

Week 3 — Deep-Dive Assessment Prep

Bit Manipulation · Advanced Strings · Linked Lists · Stack & Queue · Sliding Window & Two Pointers

Built in the same study pattern as your Week 2 guide: first-principles reasoning, dry runs, Java core logic, common traps, and practice MCQs with explanations.

1. Bit Manipulation

Why this matters: Bit operations compress useful logic into a few machine-level operations. The key is not memorizing tricks, but understanding what AND, OR, XOR and shifts do to individual binary positions.

Deep Dive — The Reasoning

Count set bits — Brian Kernighan: $n \& (n-1)$ removes the lowest set bit. Repeating until n becomes 0 means the loop runs exactly once per 1-bit.

Single number via XOR: XOR has $a \wedge a = 0$ and $a \wedge 0 = a$. Since XOR is associative and commutative, duplicate pairs cancel regardless of order.

Power of 2: every positive power of two has exactly one set bit. Therefore clearing its lowest set bit produces zero: $n > 0 \&\& (n \& (n-1)) == 0$.

Subsets via bitmask: n elements give 2^n subsets. A mask from 0 to 2^n-1 encodes inclusion: bit $i = 1$ means include `nums[i]`.

Dry Run

Single number, `arr = [4,1,2,1,2]`: $0^4=4$, $^1=5$, $^2=7$, $^1=6$, $^2=4$. The duplicate 1s and 2s cancel, leaving 4.

Code — Core Logic

```
int countSetBits(int n) {
    int count = 0;
    while (n != 0) { n &= (n - 1); count++; }
    return count;
}

int singleNumber(int[] a) {
    int x = 0;
    for (int v : a) x ^= v;
    return x;
}

boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}

List<List<Integer>> subsets(int[] a) {
    List<List<Integer>> ans = new ArrayList<>();
    for (int mask=0; mask < (1<<a.length); mask++) {
        List<Integer> cur = new ArrayList<>();
        for (int i=0;i<a.length;i++)
            if ((mask & (1<<i)) != 0) cur.add(a[i]);
        ans.add(cur);
    }
    return ans;
}
```

Common Traps (Read Before the MCQs)

- Always guard power-of-two with $n > 0$; otherwise 0 passes the bit expression.
- Plain XOR single-number works only when exactly one value occurs once and all others occur exactly twice.
- Bitmask subsets are exponential: $O(n \cdot 2^n)$. Use only for small n .
- For large shifts in Java, use `1L << i` when an int shift may overflow.

Practice MCQs / Quick Checks

Q1. What does $n \& (n-1)$ do?

Answer: Clears the lowest set bit.

Q2. Why does XOR find the single number?

Answer: Equal pairs cancel because $a \oplus a = 0$, leaving the unpaired value.

Q3. How many subsets exist for n elements?

Answer: 2^n , represented by masks 0 through $2^n - 1$.

2. Advanced String Algorithms

Why this matters: These problems test whether you can choose the right string pattern: expand around a center, prefix-function preprocessing, or a variable sliding window with frequency constraints.

Deep Dive — The Reasoning

Longest palindromic substring: every palindrome has a center. For each index, expand for odd length (i,i) and even length ($i,i+1$). This avoids checking every substring and runs in $O(n^2)$ time with $O(1)$ extra space.

KMP pattern search: when a mismatch occurs, do not restart from scratch. The LPS array stores the length of the longest proper prefix that is also a suffix, telling the pattern where to resume. Preprocessing $O(m)$, search $O(n)$, total $O(n+m)$.

Minimum window substring: keep counts of required characters, expand right until the window is valid, then shrink left while validity remains. Track the smallest valid window. This is $O(n+m)$ because each pointer moves forward at most n times.

Dry Run

Minimum window: $s=ADOBECODEBANC$, $t=ABC$. Expand until ADOBEC contains A,B,C. Shrink where possible. Later BANC is valid and shorter, so answer = BANC.

Code — Core Logic

```
String longestPalindrome(String s) {
    int start=0, end=0;
    for (int i=0;i<s.length();i++) {
        int a=expand(s,i,i), b=expand(s,i,i+1);
        int len=Math.max(a,b);
        if (len > end-start+1) {
            start=i-(len-1)/2; end=i+len/2;
        }
    }
    return s.substring(start,end+1);
}

int expand(String s,int l,int r){
    while(l>=0 && r<s.length() && s.charAt(l)==s.charAt(r)){l--;r++;}
    return r-l-1;
}

int[] buildLPS(String p){
    int[] lps=new int[p.length()];
    for(int i=1,len=0;i<p.length();){
        if(p.charAt(i)==p.charAt(len)) lps[i++]=++len;
        else if(len>0) len=lps[len-1];
        else lps[i++]=0;
    }
    return lps;
}
```

Common Traps (Read Before the MCQs)

- Palindrome has two center types: odd and even. Forgetting even centers misses answers like 'abba'.
- In KMP mismatch handling, use $lps[j-1]$; do not blindly reset j to 0.
- Minimum-window validity depends on required frequencies, not merely whether each character appears once.
- Do not rebuild substring objects inside every sliding-window step if only indices are needed.

Practice MCQs / Quick Checks

Q1. Why is KMP faster than naive matching?

Answer: It uses LPS information to avoid rechecking characters already known to match.

Q2. What are the two palindrome centers?

Answer: (i,i) for odd length and ($i,i+1$) for even length.

Q3. When do we shrink a minimum window?

Answer: Whenever the current window satisfies all required character frequencies.

3. Linked List Fundamentals

Why this matters: Linked lists reward pointer discipline. Most problems become simple once you state what each pointer means before changing links.

Deep Dive — The Reasoning

Reverse linked list: keep prev, curr and next. Save curr.next before redirecting it, then advance. $O(n)$ time, $O(1)$ space.

Cycle detection: Floyd slow/fast pointers. If a cycle exists, fast eventually laps slow and they meet. For cycle entry, reset one pointer to head and move both one step at a time.

Middle node: slow moves 1 step, fast moves 2. When fast reaches the end, slow is at the middle.

Merge two sorted lists: use a dummy node and repeatedly attach the smaller current node. This avoids special-casing the head.

Dry Run

Reverse 1→2→3: initially prev=null, curr=1. Redirect 1→null, then 2→1, then 3→2. Final head is 3, giving 3→2→1.

Code — Core Logic

```
ListNode reverse(ListNode head){
    ListNode prev=null, cur=head;
    while(cur!=null){
        ListNode next=cur.next;
        cur.next=prev;
        prev=cur; cur=next;
    }
    return prev;
}

ListNode middle(ListNode head){
    ListNode slow=head, fast=head;
    while(fast!=null && fast.next!=null){
        slow=slow.next; fast=fast.next.next;
    }
    return slow;
}

boolean hasCycle(ListNode head){
    ListNode s=head, f=head;
    while(f!=null && f.next!=null){
        s=s.next; f=f.next.next;
        if(s==f) return true;
    }
    return false;
}
```

Common Traps (Read Before the MCQs)

- In reversal, save next before changing curr.next or you lose the rest of the list.
- Cycle detection must compare node references, not merely node values.
- For even-length lists, the standard slow/fast template returns the second middle.
- Dummy nodes simplify merge logic and prevent head-edge-case bugs.

Practice MCQs / Quick Checks

Q1. Why save next during reversal?

Answer: Because curr.next is about to be overwritten.

Q2. Why does fast/slow find the middle?

Answer: Fast covers twice the distance, so when fast finishes slow has covered half.

Q3. Merge two sorted lists complexity?

Answer: $O(n+m)$ time and $O(1)$ auxiliary pointer space.

4. Advanced Linked List Concepts

Why this matters: Advanced linked-list questions combine pointer invariants with extra structure such as maps, recursion, or doubly linked lists. Drawing the links before coding is often the difference between clarity and pointer soup.

Deep Dive — The Reasoning

Reverse nodes in k-groups: first verify k nodes exist, reverse exactly k, then connect the previous group tail to the new group head. Incomplete final groups stay unchanged.

Flatten linked list: depending on the problem representation, repeatedly merge sorted child/bottom chains. The core operation is usually merge-two-sorted-lists.

Clone linked list with random pointer: $O(1)$ -extra-space method interweaves copied nodes after originals, assigns `copy.random = original.random.next`, then separates both lists.

LRU Cache: HashMap gives $O(1)$ key lookup; doubly linked list gives $O(1)$ removal/insertion. Most recently used goes near head, least recently used near tail.

Dry Run

LRU capacity 2: `put(1)`, `put(2)` → order `[2,1]`. `get(1)` → `[1,2]`. `put(3)` evicts 2 → `[3,1]`. Both lookup and recency update remain $O(1)$.

Code — Core Logic

```
// Core LRU idea
class Node { int k,v; Node prev,next; }
Map<Integer,Node> map = new HashMap<>();

void remove(Node x){
    x.prev.next=x.next;
    x.next.prev=x.prev;
}
void addFirst(Node x){
    x.next=head.next; x.prev=head;
    head.next.prev=x; head.next=x;
}
// get: lookup, remove, addFirst
// put: update existing OR insert; if over capacity remove tail.prev
```

Common Traps (Read Before the MCQs)

- For k-group reversal, never reverse a partial final group unless the problem explicitly says so.
- Random-pointer cloning needs both next and random links handled; copying only next creates a wrong graph.
- LRU requires a doubly linked list for $O(1)$ arbitrary removal.
- Always update both prev and next links when moving an LRU node.

Practice MCQs / Quick Checks

Q1. Why combine HashMap + doubly linked list in LRU?

Answer: Map gives $O(1)$ lookup; DLL gives $O(1)$ recency updates and eviction.

Q2. What happens to an incomplete final k-group?

Answer: It remains unchanged in the standard problem.

Q3. In interleaving clone, how is random copied?

Answer: `copy.random = original.random == null ? null : original.random.next`.

5. Stack & Queue Fundamentals

Why this matters: Stacks and queues are less about the container and more about recognizing order constraints: LIFO for nested/undo-style problems, FIFO for processing in arrival order.

Deep Dive — The Reasoning

Valid parentheses: push opening brackets; for each closing bracket, the stack top must be its matching opener. At the end the stack must be empty.

Stack using queues: with one queue, after pushing x , rotate all older elements behind x so the newest element sits at the front.

Min-stack: keep a second stack of minimums, or store pairs (value,currentMin), so getMin is $O(1)$.

Postfix evaluation: operands are pushed. On an operator, pop right operand first, then left operand, compute left op right, and push result.

Dry Run

Postfix '2 3 1 * + 9 -': push 2,3,1; '*' gives 3; '+' gives 5; push 9; '-' computes 5-9=-4. Operand order matters.

Code — Core Logic

```
boolean isValid(String s){
    Stack<Character> st=new Stack<>();
    for(char c:s.toCharArray()){
        if(c=='('||c=='['||c=='{') st.push(c);
        else {
            if(st.isEmpty()) return false;
            char x=st.pop();
            if((c==')'&&x!='(')||(c==']'&&x!='[')||(c=='}'&&x!='{'))
                return false;
        }
    }
    return st.isEmpty();
}
// Min stack idea: push pair [value, minSoFar]
// Postfix: b=pop(); a=pop(); push(a op b);
```

Common Traps (Read Before the MCQs)

- A closing bracket with an empty stack is immediately invalid.
- Postfix subtraction/division are order-sensitive: second popped op first popped.
- A min-stack that stores only one min variable cannot restore the previous minimum after pop unless extra history is kept.
- Queue-based stack implementations differ in which operation is costly; know your chosen invariant.

Practice MCQs / Quick Checks

Q1. What order does a stack follow?

Answer: LIFO.

Q2. In postfix, why pop right operand first?

Answer: Because the top is the most recently seen operand, which is the right operand.

Q3. How can getMin be $O(1)$?

Answer: Maintain minimum history in an auxiliary stack or store min-so-far with each entry.

6. Sliding Window & Two Pointers

Why this matters: These techniques replace repeated rescanning with moving boundaries. The central question is: what information must be maintained so the window can be updated when one element enters or leaves?

Deep Dive — The Reasoning

Longest substring without repeating: maintain a window with unique characters. With a last-seen map, jump left to $\max(\text{left}, \text{lastSeen}[c]+1)$ when a duplicate appears. $O(n)$.

Minimum window substring: expand until all required frequencies are satisfied, then shrink greedily while still valid. Record the best valid window.

Subarray sum = k: for arbitrary integers, use prefix sum + HashMap, not a standard sliding window. If current prefix is sum, any earlier prefix sum-k forms a subarray totaling k.

Two pointers: often used on sorted arrays or from opposite ends when moving a pointer has a predictable effect on the condition, such as pair sum or palindrome checking.

Dry Run

Longest unique substring, 'abcabcbb': grow 'abc' length 3. Next 'a' was last at 0, so left jumps to 1. Continue similarly; maximum remains 3.

Code — Core Logic

```
int lengthOfLongestSubstring(String s){
    Map<Character,Integer> last=new HashMap<>();
    int left=0,best=0;
    for(int right=0;right<s.length();right++){
        char c=s.charAt(right);
        if(last.containsKey(c))
            left=Math.max(left,last.get(c)+1);
        last.put(c,right);
        best=Math.max(best,right-left+1);
    }
    return best;
}

int subarraySum(int[] a,int k){
    Map<Integer,Integer> freq=new HashMap<>();
    freq.put(0,1);
    int sum=0,ans=0;
    for(int x:a){
        sum+=x;
        ans+=freq.getOrDefault(sum-k,0);
        freq.put(sum,freq.getOrDefault(sum,0)+1);
    }
    return ans;
}
```

Common Traps (Read Before the MCQs)

- Do not move left backward in longest-unique-substring; use $\max(\text{left}, \text{lastSeen}+1)$.
- Sliding window for exact sum generally fails with negative numbers because expanding/shrinking is not monotonic.
- For subarray sum k, initialize prefix frequency with $0 \rightarrow 1$ to count subarrays starting at index 0.
- Minimum window needs frequency counts, not a simple Set, because t may contain repeated characters.

Practice MCQs / Quick Checks

Q1. Why use prefix sums for subarray sum k with negatives?

Answer: Negative values destroy the monotonic behavior needed for ordinary sliding windows.

Q2. Why initialize $\text{freq}[0]=1$?

Answer: It represents the empty prefix and counts valid subarrays beginning at index 0.

Q3. Longest unique substring time complexity?

Answer: $O(n)$, because right moves once and left never moves backward.

Final Pattern Map

Problem clue	Default pattern
Duplicates cancel / binary property	XOR or bit manipulation
Palindrome substring	Expand around center
Pattern matching without rechecking	KMP + LPS
Reverse / middle / cycle	Pointer manipulation + slow/fast
O(1) cache lookup + recency	HashMap + doubly linked list
Nested matching symbols	Stack
Longest/shortest contiguous window	Sliding window
Exact subarray sum with negatives	Prefix sum + HashMap